React useContext Hook Tutorial

Introduction

The `useContext` hook is a React hook that allows you to consume context values directly in functional components. Context provides a way to pass data through the component tree without having to pass props down manually at every level. This is especially useful for global data like themes, user authentication, or language preferences.

What is Context?

Before diving into `useContext`, let's understand React Context:

- **Context** is a way to share values between components without explicitly passing props through every level
- **Context Provider** supplies the context value to its children
- **Context Consumer** accesses the context value (this is where `useContext` comes in)

Basic Usage

Here's how you create and use context with the `useContext` hook:

Step 1: Create a Context

```
import React, { createContext } from 'react';

// Create a context with a default value
const ThemeContext = createContext('light');
```

Step 2: Provide the Context

```
import React, { createContext, useState } from 'react';

const ThemeContext = createContext();

function App() {
  const [theme, setTheme] = useState('light');

  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      <Header />
      <MainContent />
    </ThemeContext.Provider>
  );
}
```

Step 3: Consume the Context

```
import React, { useContext } from 'react';

function Header() {
  const { theme, setTheme } = useContext(ThemeContext);

  return (
    <header style={{
      backgroundColor: theme === 'light' ? '#fff' : '#333',
      color: theme === 'light' ? '#333' : '#fff'
    }}>
      <h1>My App</h1>
      <button onClick={() => setTheme(theme === 'light' ? 'dark' : 'light')}>
        Toggle Theme
      </button>
    </header>
  );
}
```

Complete Example: Theme Provider

Here's a complete working example:

```
import React, { createContext, useContext, useState } from 'react';

// Step 1: Create Context
const ThemeContext = createContext();

// Step 2: Create a Provider Component
function Component1() {
  const [user, setUser] = useState("Linus");

  return (
    <ThemeContext.Provider value={user}>
      <h1>{`Hello ${user}!`}</h1>
      <Component2 />
    </ThemeContext.Provider>
  );
}

// Step 3: Components that consume the context
function Component2() {
  return (
    <>
      <h1>Component 2</h1>
      <Component3 />
    </>
  );
}
function Component3() {
  const user = useContext(ThemeContext);
  return (
    <>
      <h1>Component 3</h1>
      <h2>{`Hello ${user} again!`}</h2>
    </>
  );
}
// Step 4: Main App Component
export function App() {
  return (
    <Component1>
      <div>
        <Component3 />
      </div>
    </Component1>
  );
}
```

Explanation: - `ThemeProvider` wraps components that need access to theme data - `useContext(ThemeContext)` gives components direct access to theme values - No need to pass props through intermediate components

Multiple Contexts Example (optional)

You can use multiple contexts in the same application:

```
import React, { createContext, useContext, useState } from "react";

// User Context
const UserContext = createContext();
const ThemeContext = createContext();

function UserProvider({ children }) {
  const [user, setUser] = useState({ name: "John Doe", isLoggedIn: true
});

  return (
    <UserContext.Provider value={{ user, setUser }}>
      {children}
    </UserContext.Provider>
  );
}

function ThemeProvider({ children }) {
  const [theme, setTheme] = useState("light");

  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}

function UserProfile() {
  const { user } = useContext(UserContext);
  const { theme } = useContext(ThemeContext);

  return (
    <div
      style={{
        backgroundColor: theme === "light" ? "#fff" : "#333",
        color: theme === "light" ? "#333" : "#fff",
        padding: "1rem",
      }}
```

```
    >
      <h2>User Profile</h2>
      <p>Name: {user.name}</p>
      <p>Status: {user.isLoggedIn ? "Logged In" : "Logged Out"}</p>
    </div>
  );
}

function App() {
  return (
    <UserProvider>
      <ThemeProvider>
        <UserProfile />
      </ThemeProvider>
    </UserProvider>
  );
}

export default App;
```

In this React application, `children` is a special prop that represents whatever components are nested inside another component. It allows you to create wrapper components that can contain other elements.

The Role of `children`

Think of the `UserProvider` and `ThemeProvider` components as gift boxes. Their job is to provide a "gift" (the context data) to everything inside them. The `children` prop is what you put inside the box.

- **In `<UserProvider>`**: The `children` prop is everything inside it, which is the `<ThemeProvider>` component (and its own child, `<UserProfile />`). `UserProvider` wraps these children with the `UserContext.Provider`, making the `user` state available to them.

JavaScript

```
// Here, <ThemeProvider> and its contents are the "children"
<UserProvider>
  <ThemeProvider>
    <UserProfile />
  </ThemeProvider>
</UserProvider>
```

- **In `<ThemeProvider>`**: The `children` prop is the `<UserProfile />` component. `ThemeProvider` wraps this child with the `ThemeContext.Provider`, making the `theme` state available to it.

JavaScript

```
// Here, <UserProfile /> is the "children"
<ThemeProvider>
    <UserProfile />
</ThemeProvider>
```

In short, the `{children}` in the provider components acts as a **placeholder**. It tells React, "Render whatever components were passed inside me here." This allows the providers to share their state (like user data or theme settings) with any component nested within them, no matter how deep the nesting goes.

Custom Hook Pattern

You can create custom hooks to make context usage more convenient:

```
import React, { createContext, useContext, useState } from 'react';

const AuthContext = createContext();

// Custom hook for using auth context
function useAuth() {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error('useAuth must be used within an AuthProvider');
  }
  return context;
}

function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(false);

  const login = async (username, password) => {
    setLoading(true);
    // Simulate API call
    setTimeout(() => {
      setUser({ username, id: 1 });
      setLoading(false);
    }, 1000);
  };
```

```
  const logout = () => {
    setUser(null);
  };

  return (
    <AuthContext.Provider value={{ user, login, logout, loading }}>
      {children}
    </AuthContext.Provider>
  );
}

// Usage in components
function LoginButton() {
  const { user, login, logout, loading } = useAuth();

  if (loading) return <p>Loading...</p>;

  return user ? (
    <div>
      <p>Welcome, {user.username}!</p>
      <button onClick={logout}>Logout</button>
    </div>
  ) : (
    <button onClick={() => login('john', 'password123')}>
      Login
    </button>
  );
}
```

Exercise

Task: Create a React application with a `LanguageProvider` that: - Uses `useContext` to manage the current language (English/Spanish) - Provides translations for common words - Has components that display translated text - Includes a language switcher button

Requirements: - Create a `LanguageContext` with translations for at least 3 words/phrases - Create a `LanguageProvider` component - Create a custom `useLanguage` hook - Build a `Greeting` component that shows translated text - Build a `LanguageSwitcher` component with a toggle button

Translations to include: - English: “Hello”, “Welcome”, “Goodbye” - Spanish: “Hola”, “Bienvenido”, “Adiós”

Starter Code:

```
import React, { createContext, useContext, useState } from 'react';
```

```
const LanguageContext = createContext();

// Custom hook
function useLanguage() {
  // Your code here
}

// Provider component
function LanguageProvider({ children }) {
  // Your code here
}

// Greeting component
function Greeting() {
  // Your code here
}

// Language switcher component
function LanguageSwitcher() {
  // Your code here
}

// Main App
function App() {
  return (
    <LanguageProvider>
      <div style={{ padding: '2rem' }}>
        <LanguageSwitcher />
        <Greeting />
      </div>
    </LanguageProvider>
  );
}

export default App;
```

Expected Behavior: - Initially shows English text - Button toggles between “English” and “Español” - Greeting changes language when button is clicked - Example: “Hello, Welcome!” switches to “¡Hola, Bienvenido!”

Best Practices

✅ Do this:

```
// Create custom hooks for context
function useTheme() {
  const context = useContext(ThemeContext);
  if (!context) {
    throw new Error('useTheme must be used within ThemeProvider');
  }
  return context;
}

// Keep contexts focused on single concerns
const ThemeContext = createContext(); // Only theme-related data
const UserContext = createContext();  // Only user-related data
```

Avoid this:

```
// Don't put everything in one context
const AppContext = createContext(); // Too broad, hard to manage

// Don't forget error checking
function MyComponent() {
  const value = useContext(SomeContext); // Could be undefined
  return <div>{value.someProperty}</div>; // Potential error
}
```

Performance Considerations:

- Context triggers re-renders for all consuming components when the value changes
- Split contexts by update frequency (stable vs frequently changing data)
- Use `useMemo` for context values when needed:

```
function MyProvider({ children }) {
  const [state, setState] = useState();

  const value = useMemo(() => ({
    state,
    setState
  }), [state]);

  return (
    <MyContext.Provider value={value}>
      {children}
    </MyContext.Provider>
  );
}
```

Summary

- `useContext` allows you to consume context values directly in functional components
- Context is great for global state like themes, user auth, language preferences
- Always provide error boundaries and validation in custom hooks
- Keep contexts focused on single concerns for better maintainability
- Consider performance implications when using context for frequently changing data

For more information, see the React documentation on useContext.